NLU course project: Language Modeling

Elena Deidda (mat. 267887)

University of Trento

elena.deidda@studenti.unitn.it

1. Introduction

This project compares two strategies for word-level language modeling on the Penn TreeBank corpus, evaluated with perplexity (PPL): training a Transformer *from scratch* (Part A) versus fine-tuning a *pre-trained* model with a hand-implemented adapter (Part B). For Part A, a GPT-2-style decoder-only Transformer [1] is improved incrementally through a learning-rate and architecture search, dropout, and a weight-tying ablation, lowering test PPL from 35.68 to 33.07. For Part B, the pre-trained GPT-2 is fine-tuned with LoRA, training only low-rank adapters on the Q, K, V projections while the 124M backbone stays frozen, reaching test PPL 19.14. Both parts satisfy PPL$<$250, and Part B improves on Part A by 42% relative, training only 0.7% of the parameters.

2. Implementation details

Sentences are tokenized with the GPT-2 BPE tokenizer and padded on the right (padding ignored in the loss); PPL is the exponential of the per-token cross-entropy. Every experiment follows a greedy, one-change-at-a-time search, selected on *dev* PPL.

Part A. The search order is learning rate, then architecture (`d_model`, heads, layers, FFN size, one at a time), then dropout, then a weight-tying ablation (`lm_head` tied to the token embedding) [2]. Tying does *not* help here: the tied model needs 20 epochs to converge against 7 for the untied one, reaching a worse PPL despite $\sim$19M fewer parameters (dev PPL 44.47 vs. 36.86). Sharing the embedding and output projection forces a single matrix to serve two conflicting roles – representing an input token and scoring output tokens – which makes convergence harder, more than the smaller parameter count helps generalization. The corpus is not large enough relative to model size for overfitting to be the main concern here, so removing parameters offers little benefit. Among the other changes, increasing depth (`num_layers` 2$\rightarrow$6) gives the largest single gain in the architecture step (dev PPL 41.56$\rightarrow$37.99).

Part B. LoRA [3] is implemented manually: each attention layer injects low-rank matrices A, B into Q, K, V, computing $\Delta W x = (\alpha/r)BAx$. $A \sim \mathcal{N}(0, 1)$, B is zero-initialized, so training starts exactly from the pre-trained model; only the adapters train (0.7% of parameters at rank 16). The search order is learning rate (rank 4, α=8 fixed), then rank $r \in \{4, 8, 16\}$, then α at the best rank. PPL decreases monotonically with rank, as a larger rank gives the adapter more capacity to adjust the frozen backbone. A lower-than-default scaling (α=8 at rank 16) then gives the best result: since GPT-2 is already pre-trained on a far larger corpus than PTB, a smaller update keeps the fine-tuned model closer to the pre-trained one, preserving more of its general linguistic knowledge.

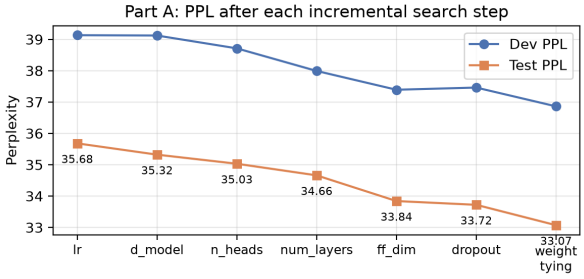

Figure 1: *Part A: dev and test PPL after each incremental search step.*

Table 1: *Part A — best model per step. Final: d*=384*, 8 heads, 6 layers, FFN 2048, dropout 0.1, no weight tying.*

Step	Config	Param	Dev	Test
0 – lr	$5{\cdot}10^{-4}$	29.2M	39.13	35.68
1.1 – d_model	384	44.6M	39.12	35.32
1.2 – heads	8	44.6M	38.71	35.03
1.3 – layers	6	47.3M	37.99	34.66
1.4 – FFN	2048	52.1M	37.39	33.84
2 – dropout	0.1	52.1M	37.46	33.72
3 – weight tying	not tied	52.1M	**36.86**	**33.07**

3. Results

Hyper-parameters are always selected on dev PPL, and test PPL is reported only for the selected configuration, to avoid test-set leakage. Table 1 and Figure 1 trace Part A: every step except weight tying improves test PPL, with depth and FFN size contributing the largest gains. Table 2 reports Part B: PPL decreases monotonically with rank, and a smaller scaling at the best rank gives the lowest test PPL, 19.14. Comparing the two parts, Part B improves on the best from-scratch model by 42% relative while training only 0.88M of 125M parameters: this gap is attributable to the pre-trained backbone already encoding general English syntax and semantics from a far larger corpus, with LoRA only specializing that knowledge to PTB, rather than to the fine-tuning technique itself.

4. References

[1] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention is all you need," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.

[2] O. Press and L. Wolf, "Using the output embedding to improve language models," in *Proceedings of the 15th Conference of the European Chapter of the Association for Computational Linguistics (EACL)*, 2017.

Table 2: *Part B — LoRA fine-tuning (lr=$5{\cdot}10^{-4}$). 124.4M backbone frozen throughout.*

Rank r	α	Train.	Dev	Test
4	8	0.22M	23.27	21.02
8	16	0.44M	21.99	19.91
16	32	0.88M	21.20	19.35
16	8	0.88M	**21.03**	**19.14**

[3] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, "LoRA: Low-rank adaptation of large language models," in *International Conference on Learning Representations (ICLR)*, 2022.

Declaration of AI use: a generative-AI assistant was used to help refactor the experiment scripts and to edit the language of this report. All architectural choices, experiments and results are the author's own.